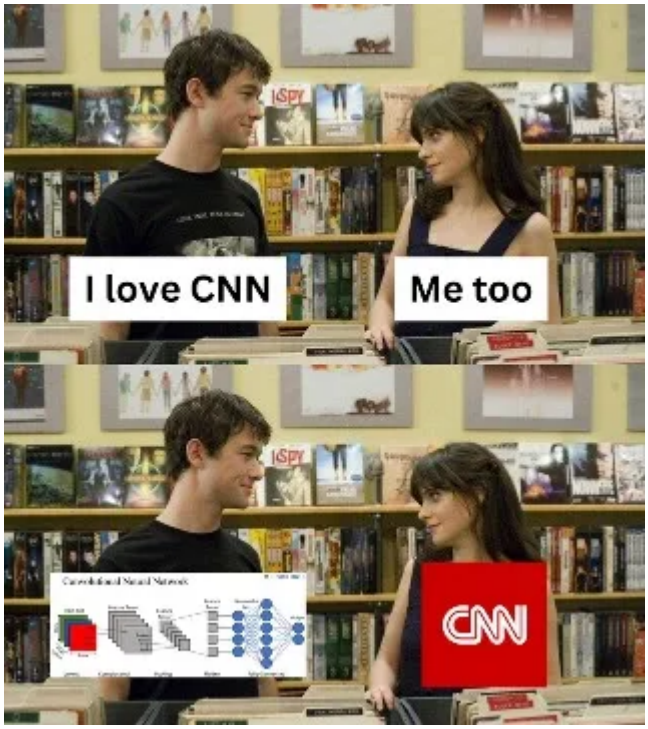
A Visual Introduction to Convolutional Neural Networks

Build Intuition Behind Convolutional Neural Network (CNN)

9 min read · 10 hours ago

Sayemuzzaman Siam



We are not the same!

Introduction

In this blog post, we'll explore what a **Convolutional Neural Network (CNN)** is and build a clear **intuitive understanding** of how it works, without diving deep into the complex math behind it. CNNs are the reason why computers can recognize faces, read license plates, detect tumors, and even drive cars.

Let's break it down in a fun way. I will use some real-life analogies and visuals to make things easier. As the saying goes, a picture is worth a thousand words.

What Is a Convolutional Neural Network

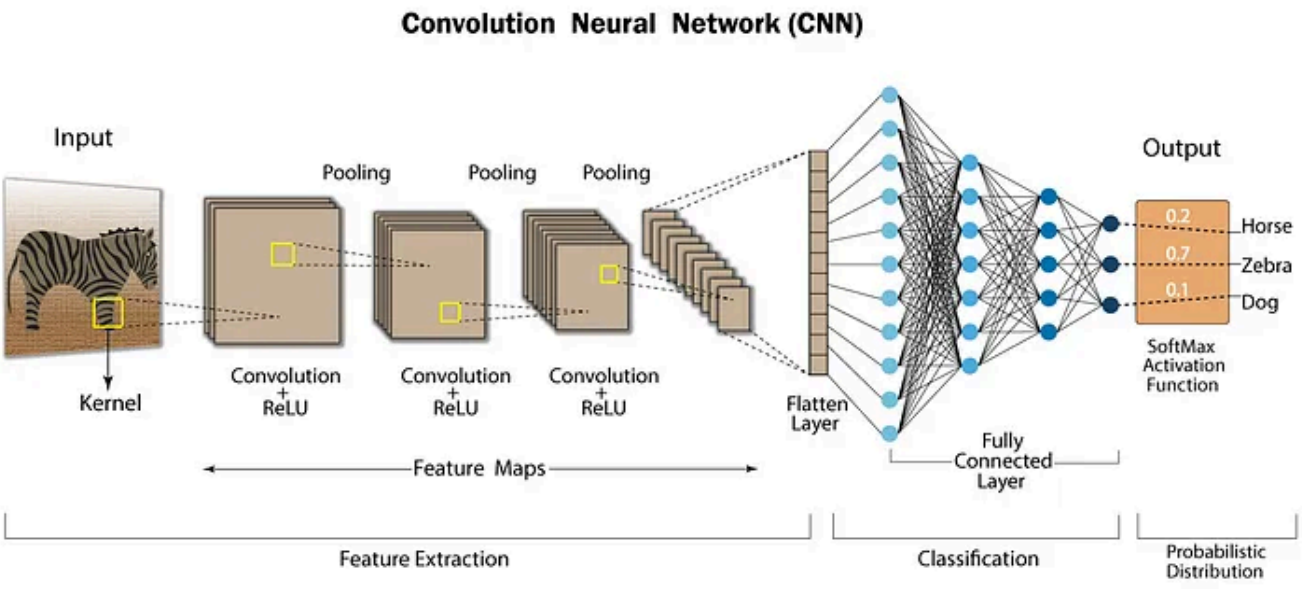
A **Convolutional Neural Network (CNN)** is a **special type of neural network** designed to process and analyze **visual data** — like images and videos.

Think of a CNN as the **Eye of a Computer**. It helps computer to Recognize objects in an image, Classify images, Segment images, Detect object positions etc.

CNNs are inspired by how our brain and eyes work together to process what we see.

How CNN Works

To understand how CNNs work, let's walk through each component in a typical CNN architecture step by step. Think of each step as part of a visual processing pipeline, where the machine gradually builds a deeper understanding of the image.



Convolution Neural Network Architecture

Before we dive into the detailed workings of the architecture, let's see a **high-level overview** of the **CNN architecture**. The Image above shows a generalized version of CNN architecture. This architecture is typically divided into **three** main sections.

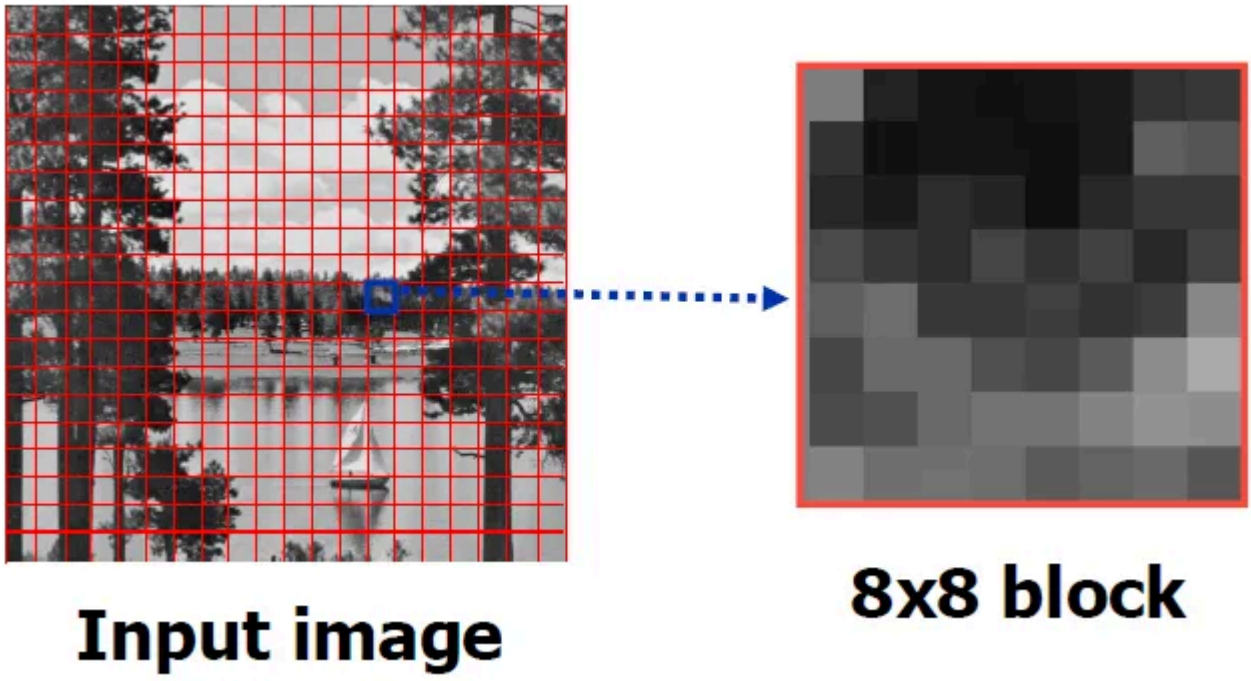
1. **Feature Extraction :** This includes layers from the **Input** to the **Flatten Layer**.
2. **Classification :** This involves the **Fully Connected(Dense) Layer**.

3. **Probabilistic Distribution :** This is the **final output layer**, which produces a probability score for each class.

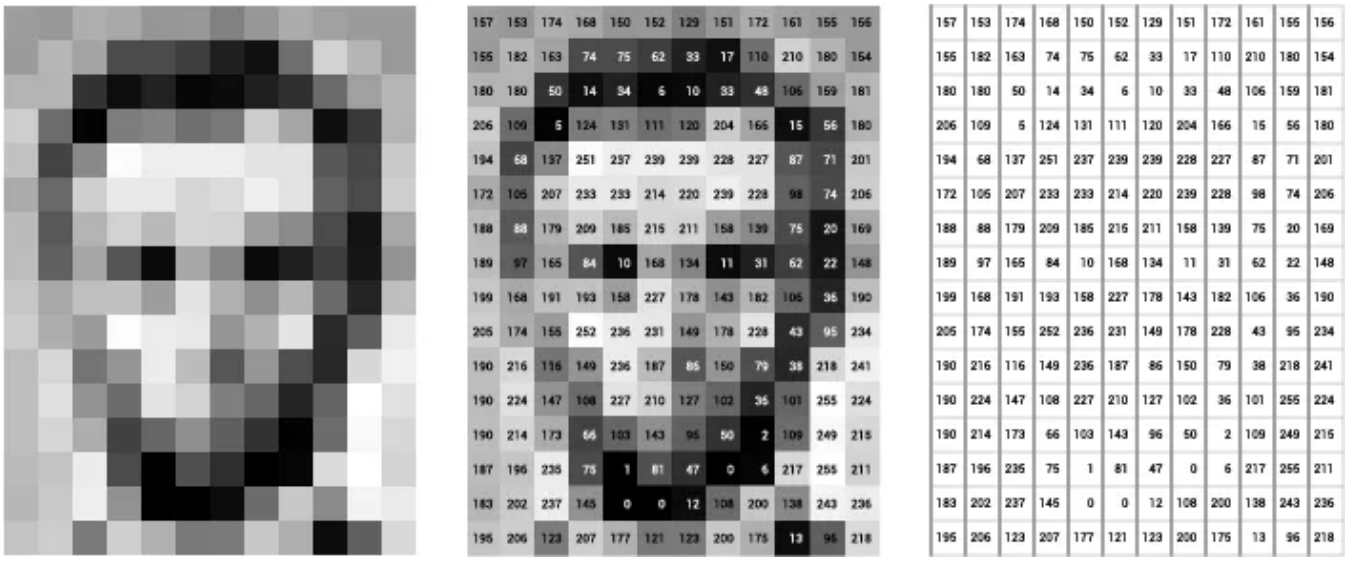
Well congratulations, you have a high-level overview understanding of CNN Architecture. Now let's dive into the details step by step. **Ready to go? Let's begin.**

Step 1: Input Layer — How Computers See Images

In CNN our Input is an Image. Now in the Input Layer, computer doesn't see a beautiful photo. Instead, it sees a **grid of numbers.** Every image is stored as **pixels,** tiny squares — each with color values. A **grayscale image** is stored as a **2D matrix** with dimensions *(rows* x *columns)*, where each value represents the intensity of light (from black=0 to white=255).



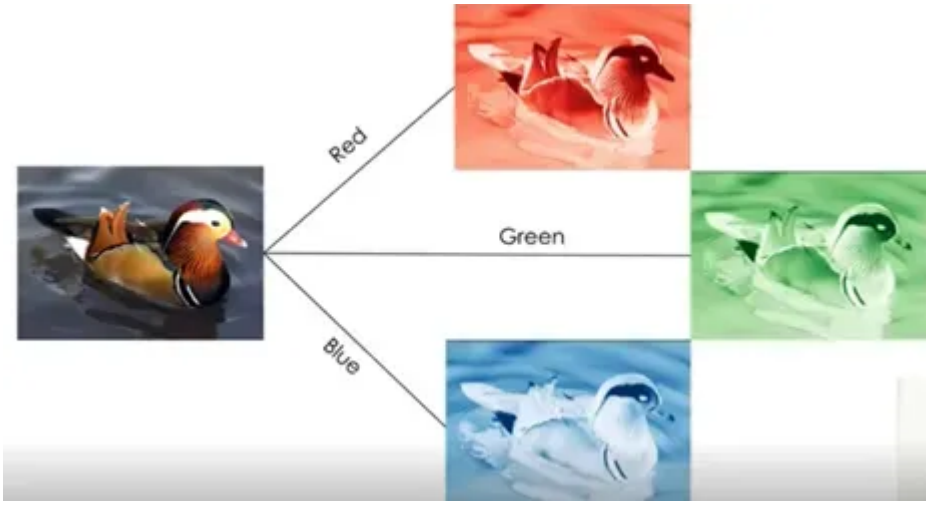
If you zoom in on a grayscale image, you will see something like this.



Here you can see how a computer actually stores an image. Ta-da! It's all numbers!

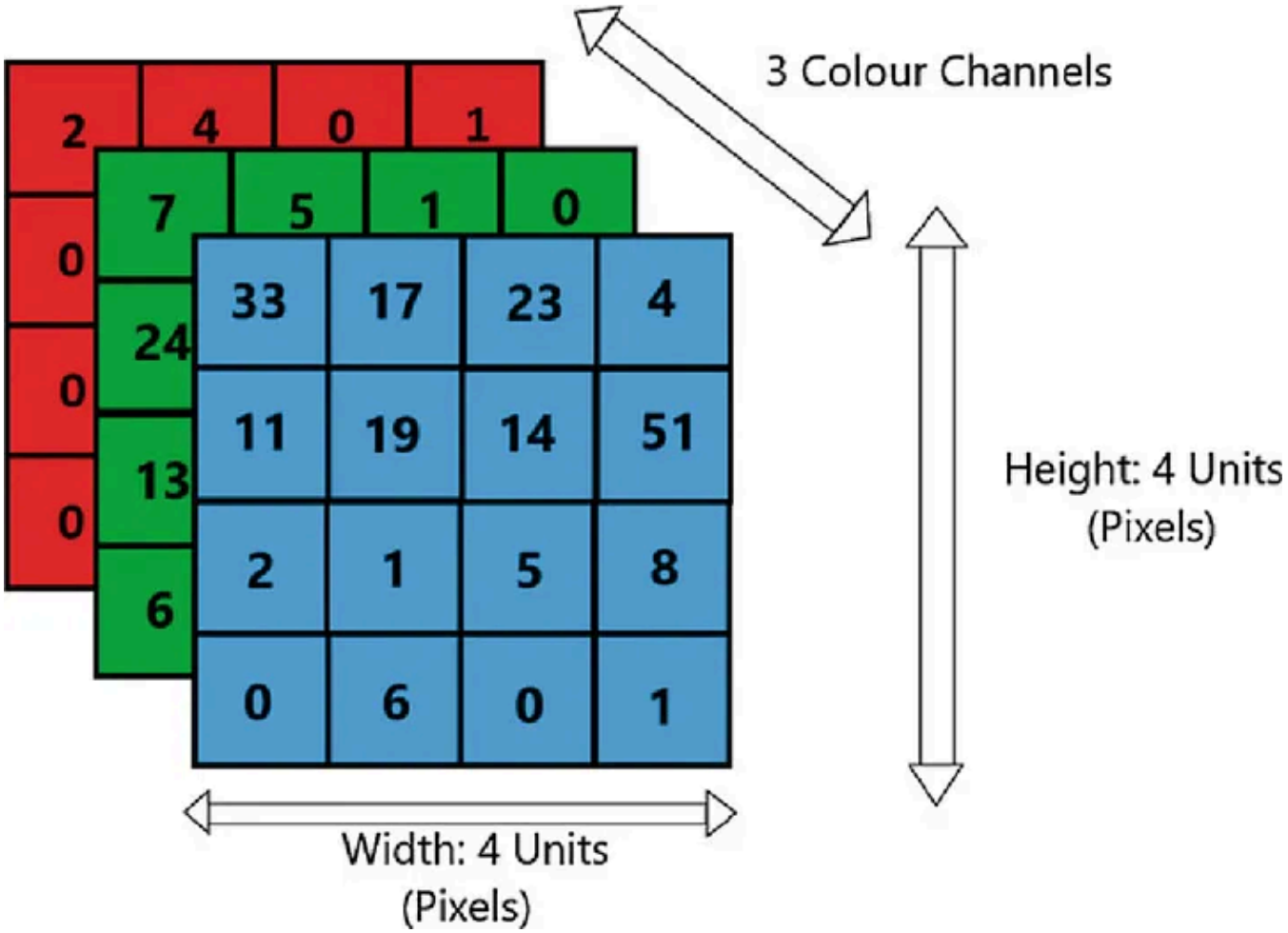
These two images above shows how gray scale images are stored in computer.

For a **color image**, things get a bit more complex. Each pixel contains **three values** — Red, Green, and Blue. These are the **primary colors** of light. By adjusting the intensity of each of these three components, we can represent virtually any color.



How Colored Images Are Made Using RGB Channels!

We don't mix these colors like we mix paints. Instead, we use separate layers, called **channels**. A color image is stored as a **3D array** with dimensions *(rows* x *columns* x *3)*, where the third dimension corresponds to the **RGB channels:** one for Red, one for Green, and one for Blue.



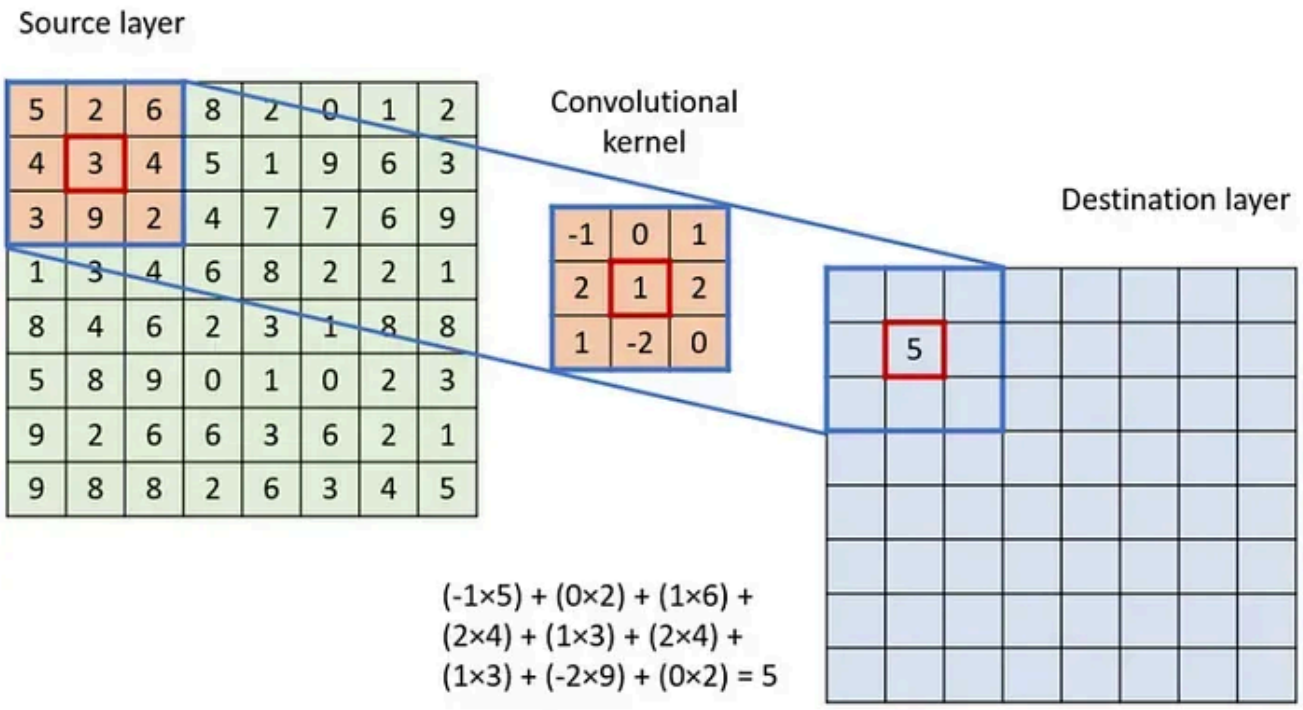
This image shows how RGB (colored) images are stored. It represents a 4×4×3

So now we understand how a computer sees and stores images, both in grayscale and in RGB format. This is the end of **Step 1:** Input and Input Layer in a CNN.

Next, let's move on to what happens after the image is received and how the network begins to **extract features** using filters/kernels in the next layer.

Step 2: Convolution Operation— The Feature Finder

The heart of CNN is the **convolution operation.** It uses a small filters also known as **kernels** (typically of size 3×3 or 5×5) that slides across the image to detect patterns such as edges, textures, or corners. As shown in the Image below:



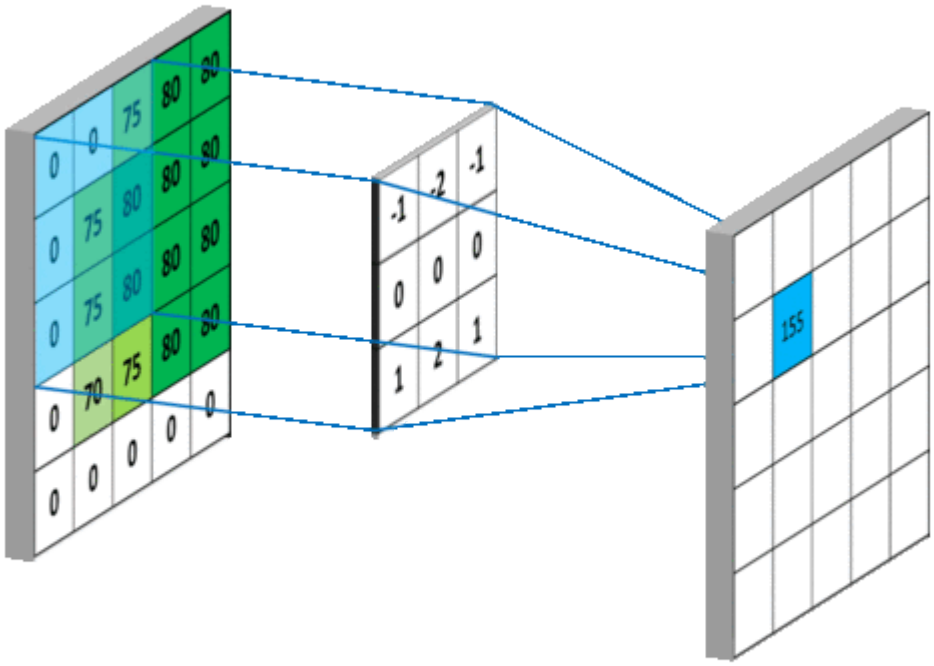
Convolution Operation: Applying a Kernel to an Image

At each position, the kernel multiplies its values (weights) with the corresponding image pixel values and sums up the result. This process creates a new matrix called the **feature map**, which highlights specific features in the image. Each filter is designed to detect **a particular feature**, such as edges, corners, or textures.

Two important concepts in the convolution operation:

- **Padding**: Sometimes, when the kernel slides over the image, it can miss information at the edges. Padding is the technique of adding extra pixels (usually zeros) around the border of the image to preserve its spatial dimensions. This ensures that edge features are not lost during convolution.

- **Stride**: Stride defines **how many pixels** the kernel moves at a time. A stride of 1 means the kernel moves one pixel at a time, while a stride of 2 would mean it jumps two pixels. Larger strides reduce the output size and make the network faster, but they might skip important details.

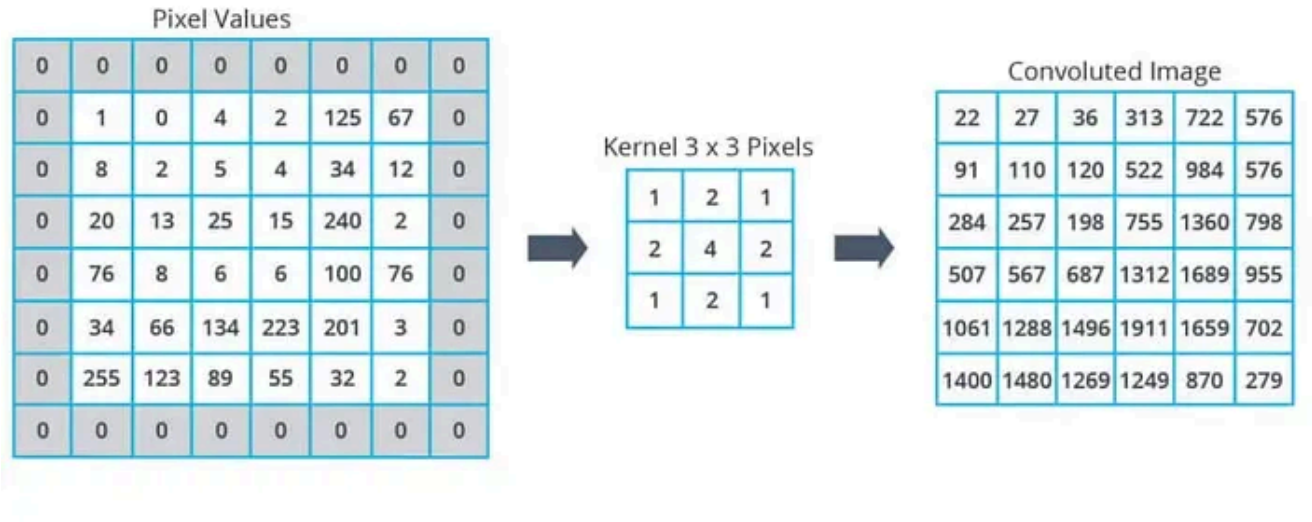


This is how the kernel slides through the whole image

Analogy: *Think of it like scanning an image using different types of magnifying glasses. Each magnifying glass is designed to highlight a specific pattern like edges, shadows, or color contrasts. That's exactly what a kernel does: it focuses on specific features of the image.*

Step 3: Feature Maps

The result of the convolution operation is a **feature map**, where important features from the image are highlighted, and irrelevant parts are reduced. These maps form the foundation of **feature extraction.**



Convolution Operation with Zero Padding → Convolved Image (Feature Map)

Analogy: *It's like turning a photo into a pencil sketch that shows outlines clearly and removes color distractions.*

Step 4: Activation Function — Adding Life

After the convolution operation, we apply an **activation function**, typically **ReLU (Rectified Linear Unit)**. ReLU works by **replacing all negative values with zero**, while keeping positive values unchanged.

This step introduces **non-linearity** into the model. Without it, the CNN would behave like a **simple linear system** and wouldn't be able to learn complex patterns. Non-linearity is essential for recognizing real-world features like curves, textures, and intricate shapes.

ReLU: Zero Negativity.

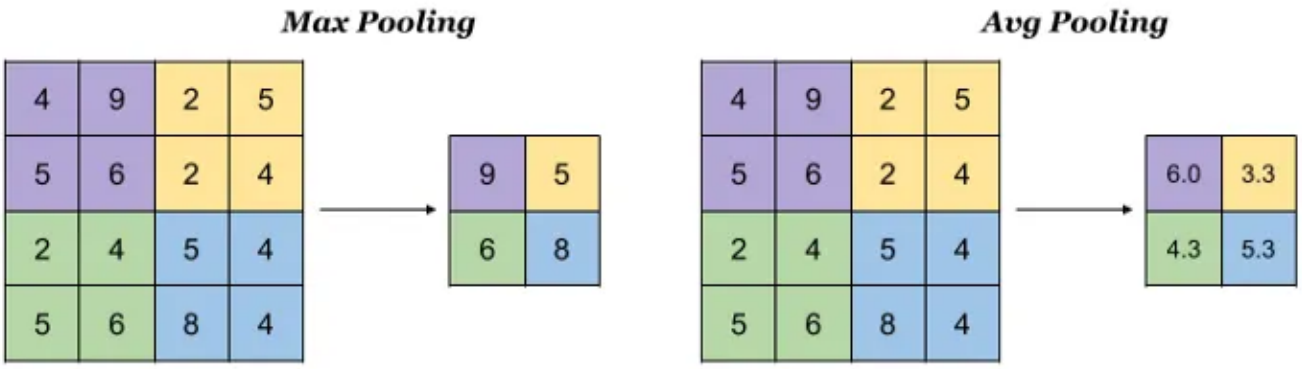
Analogy: *Imagine a machine that refuses to deal with negativity,* ***any negative value that enters gets turned into zero****. That's exactly what ReLU does. It's all about ZERO negativity!*

Other activation functions also exist (like Sigmoid or Tanh), but ReLU is widely used in CNNs because it's simple and efficient . It helps models train faster and avoids issues like vanishing gradients.

Now let's move to the next step.

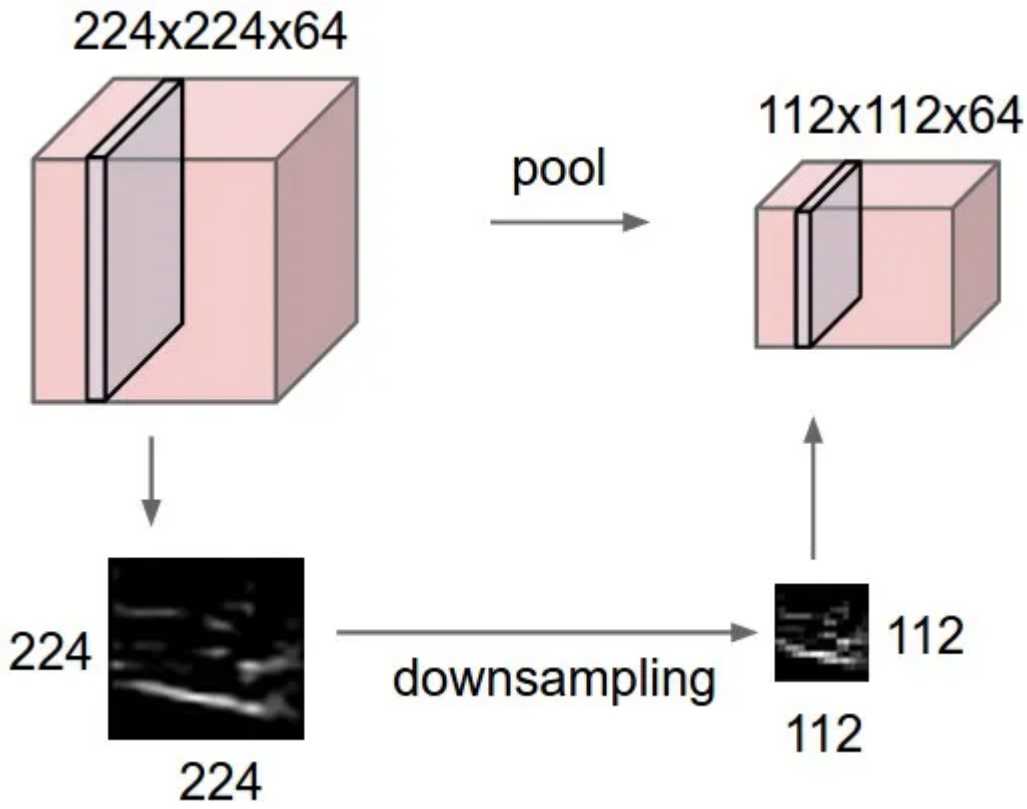
Step 5: Pooling — Zooming Out to See the Bigger Picture

After applying convolution and activation, the next step is pooling, usually max pooling. Pooling is used to **reduce the size of the feature map**, which helps decrease computation and prevents overfitting , **while still retaining the most important information**.



Max Pooling and Average Pooling

In max pooling, the feature map is divided into small blocks (like 2×2), and for each block, only the **maximum value** is kept. This process keeps the strongest features and discards weaker ones.



Pooling: Shrinking the Image While Keeping the Important Features

Analogy: *Think of it like* ***zooming out on a photo,*** *you might lose tiny details, but you still retain the* ***essential structure*** *and patterns. It's a way of simplifying the view while preserving the important information.*

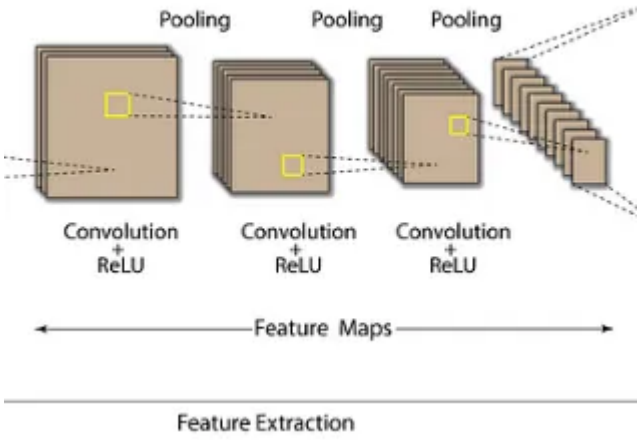
Step 6: Stacking Layers — Deep Feature Extraction

In a real Convolutional Neural Network, we don't stop after just one round of convolution, activation, and pooling. Instead, we **stack multiple layers** of these

operations to build a deep architecture.
This repeated process allows the network to **learn complex features** from the image.

Here's what happens at each level:

- **Early layers** — detect simple features like **edges and lines**
- **Middle layers** — capture more complex patterns like **shapes, textures, or corners**
- **Deeper layers** — identify high-level features like **object parts or even entire objects**



Repeat the same step

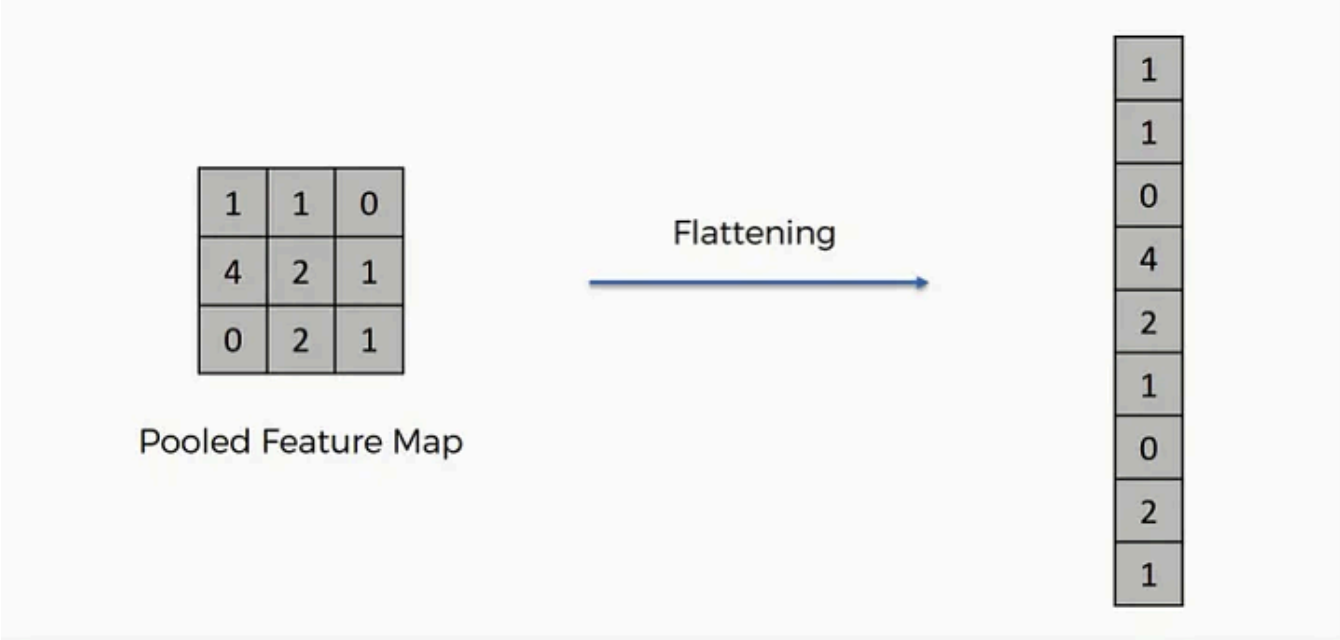
Each layer builds on the one before, gradually transforming raw pixels into meaningful representations the model can understand.

Analogy: *It's like reading, first you recognize letters, then words, then sentences, then meaning. CNNs do the same thing from simple patterns to high-level understanding.*

This **layered approach** is why CNNs are so powerful in tasks like image classification, object detection, and facial recognition. Now let's move onto the next step.

Step 7: Flatten Layer — Preparing for Decision

After all the convolutional and pooling layers have extracted meaningful features, the next step is to flatten the data. The flatten layer takes the multi-dimensional feature maps (like a **3D** cube of numbers) and converts them into a **1D vector** — a single, long list of numbers.



This is how the feature map is flattened

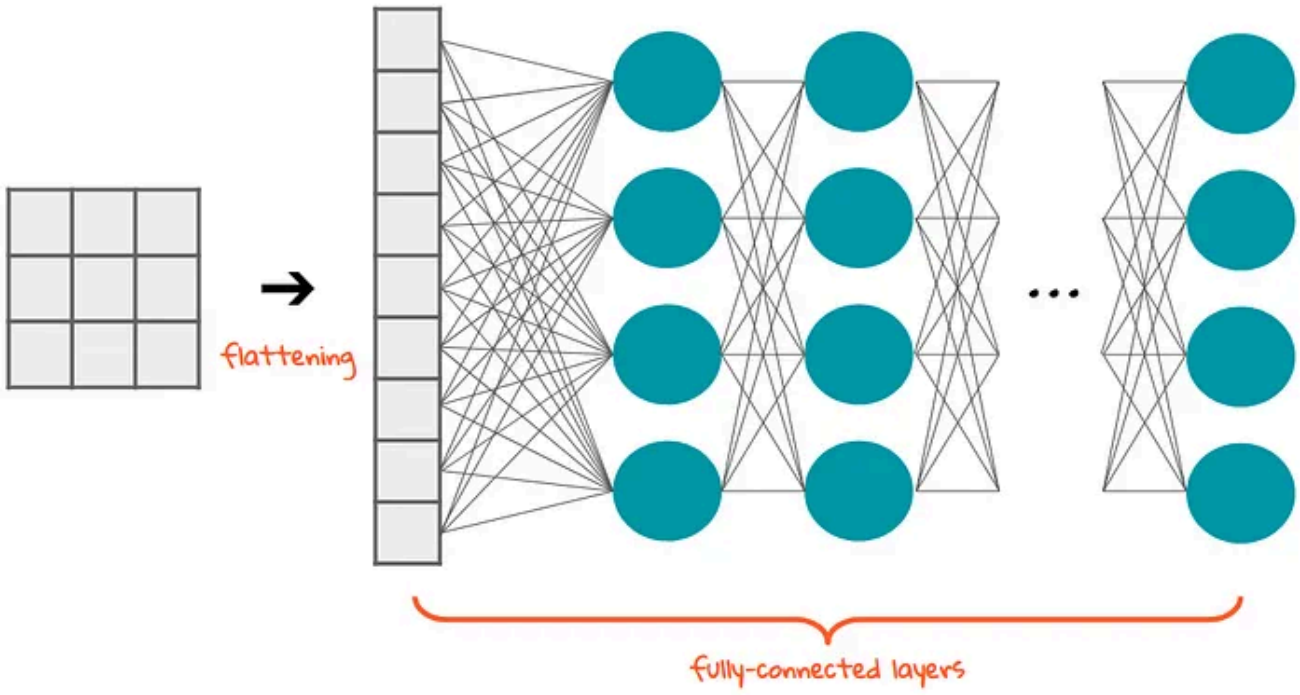
This transformation is necessary because the next layer is the fully **connected (dense) layer,** it expects a flat input, similar to how traditional neural networks operate.

Analogy: *Imagine putting different chunks of fruits (shapes, sizes, colors) into a blender. The blender doesn't care about their structures. It turns everything into a smooth juice that's ready to drink. Similarly, flattening blends all the structured features into a single vector, ready to be passed into the decision-making layers.*

Step 8: Fully Connected Layer — Final Decision

Once we have a flattened vector of features, it's passed into a **fully connected layer** (also known as a **dense layer**). Here, **every neuron is connected to every feature**, just like in a traditional neural network.

This is where the model **combines all the clues** it has gathered so far and makes the final prediction — like deciding whether the image contains a Horse, a Zebra, or something else entirely.



Our Old Friend, the Dense Layer or FC layer.

In fact, this part of the CNN is just our old friend. Its the **Multilayer Perceptron (MLP)** or **Artificial Neural Network (ANN)**. Never heard of MLPs or ANNs? Don't worry, you're not alone. Just take a short field trip to the land of **basic neural networks**, pick up a few neurons, and come back smarter. We'll be right here, waiting for you.

Analogy: *Think of it like solving a mystery. You've gathered evidence from various sources (edges, shapes, textures), and now it's time to* ***put all the clues together*** *to reach a final verdict.*

Step 9: Output Layer — Final Prediction

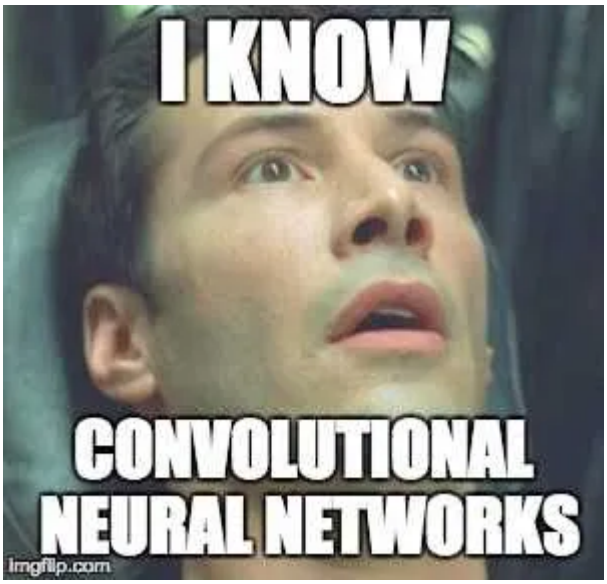
At the very end of the CNN, we have the **output layer**. This layer gives us a **confidence score** for each possible class the model has been trained on. For example:

- Horse: 0.2
- Zebra: 0.7
- Dog: 0.1

The class with the **highest score** is chosen as the model's **final prediction**.

This is where all the layers of feature extraction and pattern recognition come together to **make a decision**.

Congrats, you've now built an intuitive, layer-by-layer understanding of a **Convolutional Neural Network (CNN)**!
From raw pixels to pattern detection to final predictions, you've seen how a CNN "sees" and understands images.



Well, Congrats

Conclusion:

Keep in mind, this is a **simplified overview** of CNNs. Each step has its own math and complexities behind it. This guide is meant to help you **build your intuition** before diving into the more technical aspects.

Good luck on your deep learning journey.

Convolution Neural Net | Cnn | Deep Learning | Data Science | Computer Vision

Edit profile

Written by Sayemuzzaman Siam

1 follower · 19 following

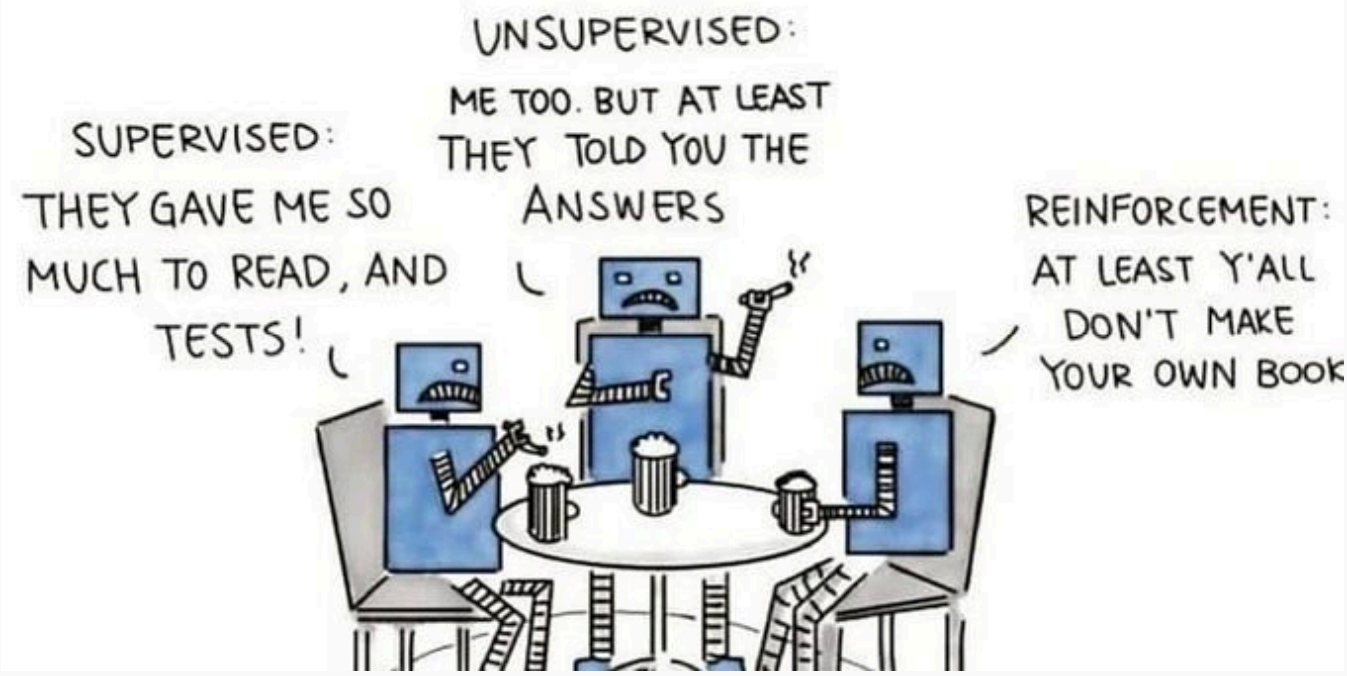
Deep Learning Enthusiast.

No responses yet

Sayemuzzaman Siam

What are your thoughts?

More from Sayemuzzaman Siam

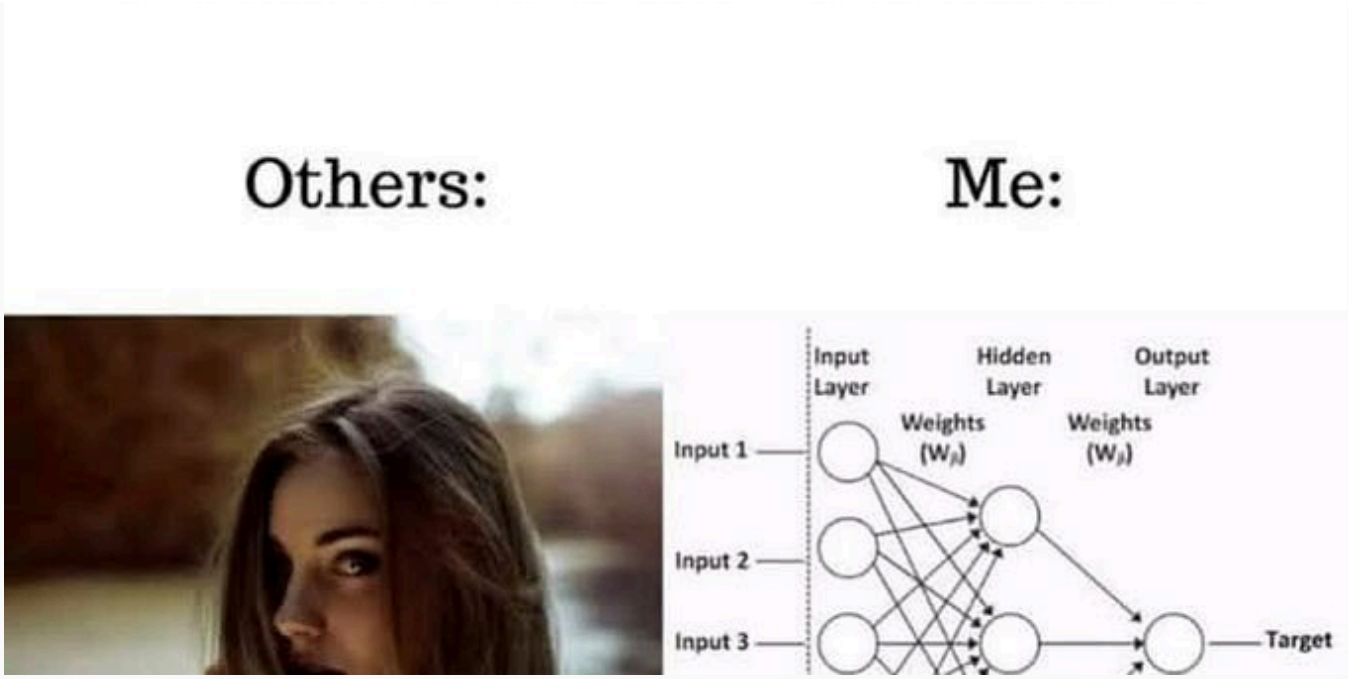


Sayemuzzaman Siam

Machine Learning Algorithms

Machine learning algorithms can be categorized into three main types:

Jan 11 1



Sayemuzzaman Siam

Check Your Model: Is It Getting It Right, or Just...

Training a machine learning model is like teaching a pet—you want it to learn the right tricks without overdoing it or forgetting the...

Jan 27

Sayemuzzaman Siam

A simple overview on Machine Learning

What is Machine Learning?